

UE23CS342BA4 : Generative AI and its Applications

Hands-on -2

Date:17/02/2026

NAME:BHAVANI	SRN:PES2UG23CS125	SEC:B
--------------	-------------------	-------

1_LangChain_Foundation

Temperature:

Setting different temp:

```
prompt = "Define the word 'Idea' in one sentence."

print("---- FOCUSED (Temp=0) ----")
print(f"Run 1: {llm_focused.invoke(prompt).content}")
print(f"Run 2: {llm_focused.invoke(prompt).content}")

--- FOCUSED (Temp=0) ---
Run 1: An idea is a thought, concept, or suggestion that is formed or exists in the mind.
Run 2: An idea is a thought, concept, or mental image formed in the mind.

print("---- CREATIVE (Temp=1) ----")
print(f"Run 1: {llm_creative.invoke(prompt).content}")
print(f"Run 2: {llm_creative.invoke(prompt).content}")

--- CREATIVE (Temp=1) ---
Run 1: An idea is a mental concept, thought, or impression that often serves as a plan, an opinion, or the foundation for understanding and action.
Run 2: An idea is a mental conception, plan, or suggestion that serves as a basis for action, understanding, or belief.
```

```
11 prompt = "Define the word 'Idea' in one sentence."
9%
print("---- FOCUSED (Temp=0) ----")
print(f"Run 1: {llm_focused.invoke(prompt).content}")
print(f"Run 2: {llm_focused.invoke(prompt).content}")

✓ --- FOCUSED (Temp=0) ---
Run 1: An idea is a thought, concept, or suggestion that is formed or exists in the mind.
Run 2: An idea is a thought, concept, or mental image formed in the mind.

33 1 print("---- CREATIVE (Temp=2) ----")
print(f"Run 1: {llm_creative.invoke(prompt).content}")
print(f"Run 2: {llm_creative.invoke(prompt).content}")

✓ ... --- CREATIVE (Temp=2) ---
Run 1: An idea is a mental concept, plan, or understanding formed in the mind as a result of thought or imagination.
Run 2: An idea is a thought, concept, or plan formed in the mind.
```

PART 1a :LangChain & Models

1. Why LangChain?

LangChain is a framework that makes it easy to use AI models.

Instead of directly connecting to Gemini or OpenAI, we use LangChain as a middle layer.

It makes our code clean and easy to change models later.

2. Tokens

AI does not read full sentences. It breaks text into small parts called **tokens**.

Tokens are important because:

- Cost depends on tokens.
- Models have a limit (context window).

3. API Key Safety

We should not write API keys directly in code.

We use environment variables or `getpass()` to keep it secure.

4. Temperature

Temperature controls creativity:

- Low (0) → Same, focused answer.
- High (1) → More creative, different answers.

PART 1b :Prompts & Parsers

1. Message Types

Instead of simple strings, we use:

- System → Instructions to AI
- Human → User message
- AI → Model reply
- System messages control behavior.

2. Prompt Templates

Instead of manually formatting strings, we use **PromptTemplate**.

It makes the code clean and safe.

3. Context Window

Each model has a memory limit.

If input is too long, old data gets removed.

4. Output Parser

Model gives output in object format.

Parser converts it into simple text.

PART 1c: LCEL

1. Problem with Manual Code

Calling prompt → model → parser separately makes code messy.

2. LCEL Solution

We use `|` operator to connect everything.

Example idea:

Prompt | Model | Parser

This makes:

- Code short
- Easy to understand
- Easy to modify

3. Main Benefit

LCEL makes AI pipeline clean and structured.

Notebook 2 : 2_prompt_engineering

PART 2a: The Anatomy of a Prompt

1. What is a Prompt?

A prompt is the instruction or question we give to the AI model.

The quality of the answer depends on how clearly we write the prompt.

Good prompt → Good output

Bad prompt → Confusing output

PART 2b: Prompt Techniques & Optimization

1. Improving Prompts

In this part, we learned how to improve prompts to get better results.

Small changes in wording can:

- Improve clarity
- Reduce wrong answers
- Make output structured

2. Chain of Thought Prompting

Instead of asking for direct answer, we tell the model:

“Think step by step.”

This helps the model:

- Break the problem into smaller parts
- Reduce mistakes in reasoning
- Give logical answers

Very useful for:

- Math problems

- Logical reasoning
- Multi-step questions

3. Role-based Prompting

We can tell the AI to act as:

- Teacher
- Lawyer
- Software engineer
- Interviewer

This changes:

- Tone
- Depth of explanation
- Style of response

4. Structured Output

Instead of normal paragraph answers, we can ask:

- “Return in bullet points”
- “Return in JSON”
- “Return in table format”

This is important for:

- Building applications
- Automation
- Data extraction

PART 2c: Advanced Prompting & Real Applications

1. Combining Techniques

In this part, we combined:

- Role
- Context
- Format instructions
- Chain-of-thought

2. Prompt Engineering for Applications

When building real apps:

- We must control output format.
- We must avoid hallucinations.
- We must give clear constraints.

Example:

Instead of “Explain AI”

We say:

“Explain AI in 5 bullet points for 10th grade students.”

3. Reducing Hallucinations

Hallucination = AI gives wrong or made-up information.

We reduce it by:

- Giving proper context
- Asking to say “I don’t know” if unsure
- Using structured prompts

Notebook 3 : 3_Advanced Prompting

PART 3a: Chain of Thought (CoT)

1. Introduction to Chain of Thought (CoT)

Chain of Thought (CoT) is a prompting technique where we ask the model to **think step-by-step before giving the final answer**.

Normally, the model gives a direct answer.

In CoT, we guide it to show its reasoning process.

This improves accuracy for reasoning problems.

2. Why Chain of Thought is Needed?

For simple questions, direct answers work fine.

But for:

- Math problems
- Logical reasoning
- Multi-step questions

The model may make mistakes if it answers directly.

By using CoT:

- The model breaks the problem into steps.

- Solves step-by-step.
- Then gives the final answer.
- This reduces errors.

3. Zero-Shot CoT

Zero-shot CoT means we simply add:

"Let's think step by step."

to the prompt.

Even this small instruction makes the model reason properly.

It improves performance without giving any examples.

4. Few-Shot CoT

In Few-shot CoT:

- We provide examples.
- Each example shows step-by-step reasoning.
- Then we give a new question.

The model learns the reasoning pattern from examples and applies it.

This usually gives better results than zero-shot.

5. Comparison: Normal vs CoT

Normal Prompt:

→ Direct answer

→ May be wrong

Chain of Thought Prompt:

→ Step-by-step explanation

→ More accurate

→ Better for complex problems

4. Summary & Comparison Table

Method	Structure	Best For...	Cost/Latency
Simple Prompt	Input -> Output	Simple facts, summaries	★ Low
CoT (Chain)	Input -> Steps -> Output	Math, Logic, Debugging	★★ Med
ToT (Tree)	Input -> 3x Branches -> Select -> Output	Strategic decisions, Brainstorming	★★★ High
GoT (Graph)	Input -> Branch -> Mix/Aggregate -> Output	Creative Writing, Research Synthesis	★★★★ V. High

Recommendation: Start with CoT. Only use ToT/GoT if CoT fails.

Notebook 4 : 4_RAG_and_Vector_Stores

Unit 2: RAG, Vector Stores, and Indexing

1. Introduction to RAG

RAG stands for **Retrieval-Augmented Generation**.

Normally, an AI model answers only based on the data it was trained on.

But sometimes:

- The information may be outdated.
- The model may not know specific private data.

RAG solves this problem.

In RAG:

1. First, we **retrieve relevant information** from our documents.
2. Then, we give that information to the model.
3. The model generates the final answer using that data.

So RAG = Retrieval + Generation.

2. Why RAG is Important?

Without RAG:

- Model may give wrong or general answers.
- Cannot access private documents.

With RAG:

- Answers are based on our own data.
- More accurate and reliable.
- Useful for chatbots using company documents.

3. What are Embeddings?

Embeddings convert text into numbers (vectors).

AI cannot understand text directly.

So text is converted into **vector form (numerical representation)**.

Similar meanings → Similar vectors.

Different meanings → Different vectors.

This helps in searching similar content.

4. What is a Vector Store?

A vector store is a database that stores embeddings.

Steps:

1. Convert documents into embeddings.
2. Store them in a vector database.
3. When user asks a question, convert question into embedding.
4. Find the most similar stored vectors.

So vector store helps in **similarity search**.

5. What is Indexing?

Indexing means organizing data in a way that makes searching faster.

In RAG:

- Documents are split into smaller chunks.
- Each chunk is converted into embeddings.
- These embeddings are stored and indexed.

This allows fast retrieval of relevant information.

6. RAG Workflow:

1. Load documents
2. Split into chunks
3. Convert chunks into embeddings
4. Store in vector database
5. User asks question
6. Retrieve relevant chunks
7. Send retrieved data + question to model
8. Model generates final answer

Unit2_Assignment_MoE

4. System Architecture

The system consists of three main components:

4.1 Router (Intent Classifier)

The Router is responsible for analyzing the user's input and classifying it into one of the following categories:

- Technical
- Billing
- Sales
- General (fallback)

The router uses:

- Temperature = 0
This ensures deterministic and consistent classification results.

4.2 Experts (Simulated using System Prompts)

Each expert is defined using a different system prompt but uses the same base model.

Technical Expert

- Code-focused
- Logical and precise
- Provides debugging steps and corrected code

Billing Expert

- Empathetic and professional
- Explains refund policies and charges clearly

Sales Expert

- Persuasive and informative
- Highlights product benefits and features

General Expert

- Friendly fallback assistant

Although the same LLM model is used, changing the system prompt modifies the behavior of the model, effectively simulating different experts.

The expert models use:

- Temperature = 0.7
This allows slightly more creative and flexible responses.

4.3 Orchestrator

The Orchestrator is the main control function that:

1. Calls the router to classify the query.
2. Selects the correct expert configuration.
3. Sends the user query with the expert's system prompt.
4. Returns the final generated response.

This demonstrates the working principle of Mixture of Experts architecture where routing logic dynamically selects specialized sub-models.

5. Bonus: Tool Use Expert

As part of the bonus challenge, a tool-based expert was implemented.

If the user asks for:

“current price of Bitcoin”

Instead of calling the LLM, the system directly calls a Python function that returns mock data.

This demonstrates:

- Hybrid AI systems
- Integration of external tools
- Bypassing LLM when structured data is needed

This is similar to how modern AI systems integrate APIs and tools.

6. Example Working

Example 1 – Technical Query

Input:

My Python script is throwing an IndexError.

Router Output: technical

The system loads the Technical Expert system prompt and provides debugging guidance.